

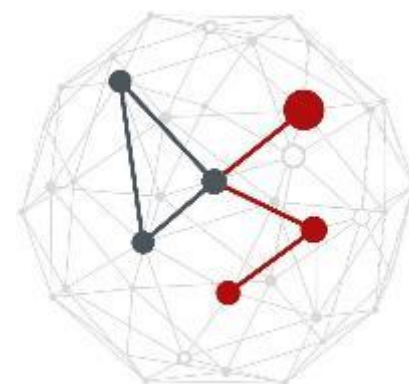
LAB 5: CNN-RNN CLASSIFIER

Eleonora Cicciarella, Cesare Bidini

eleonora.cicciarella@phd.unipd.it

cesare.bidini@phd.unipd.it

University of Padova, IT



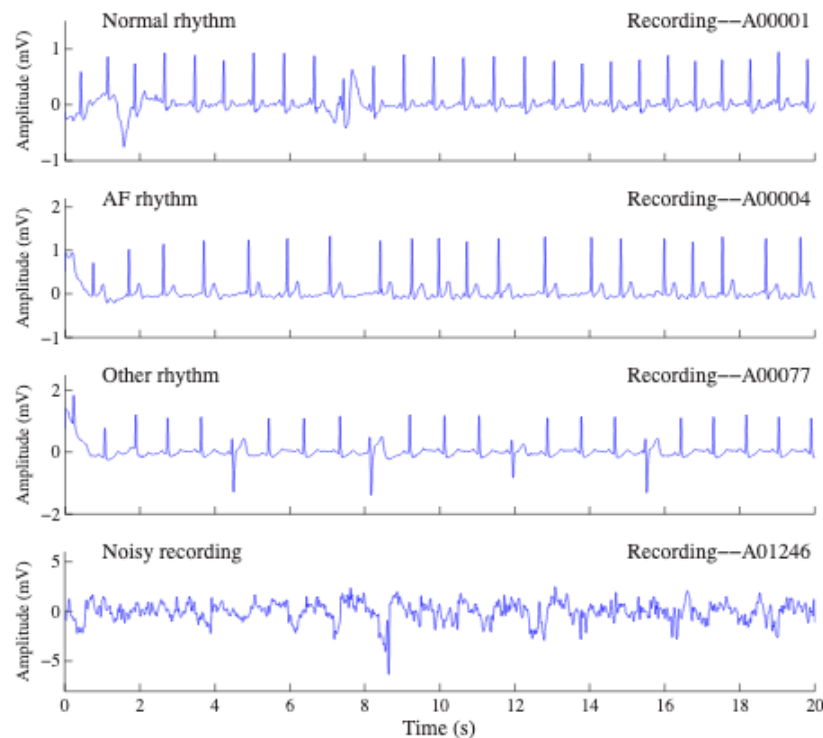
Lab 5

- **CNN-RNN classifier**

- TensorFlow/Keras
- **The challenge:**
 - abnormal heart rhythm recognition

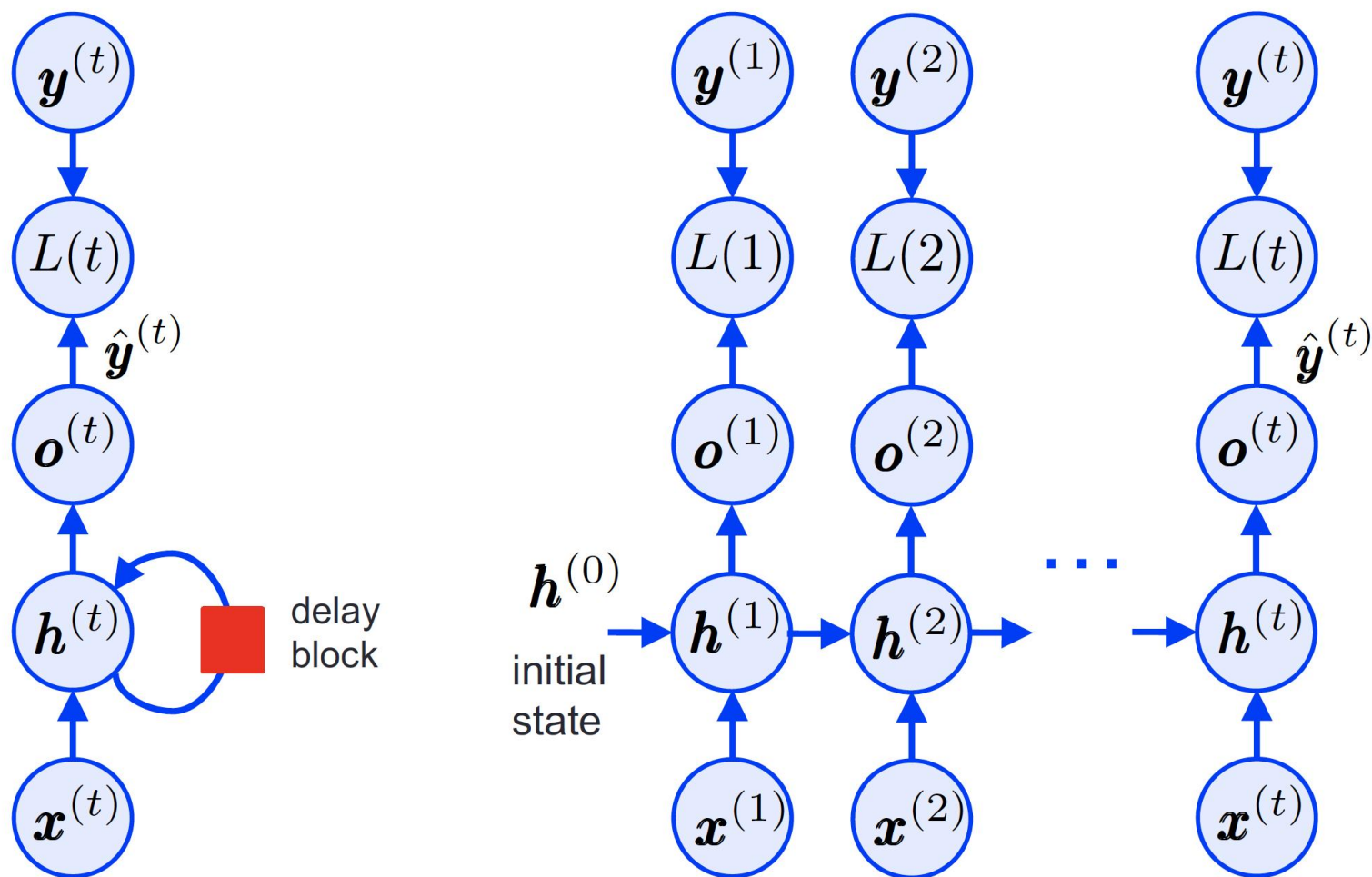
- **You will learn to:**

- use **Pandas** Python library to handle data structures in Python
- use **Scikit-learn** and **SciPy** Python libraries
- preprocess the input data
- implement a **CNN-RNN** based classifier using TensorFlow **Keras**
- train the classifier and test its performance using different metrics

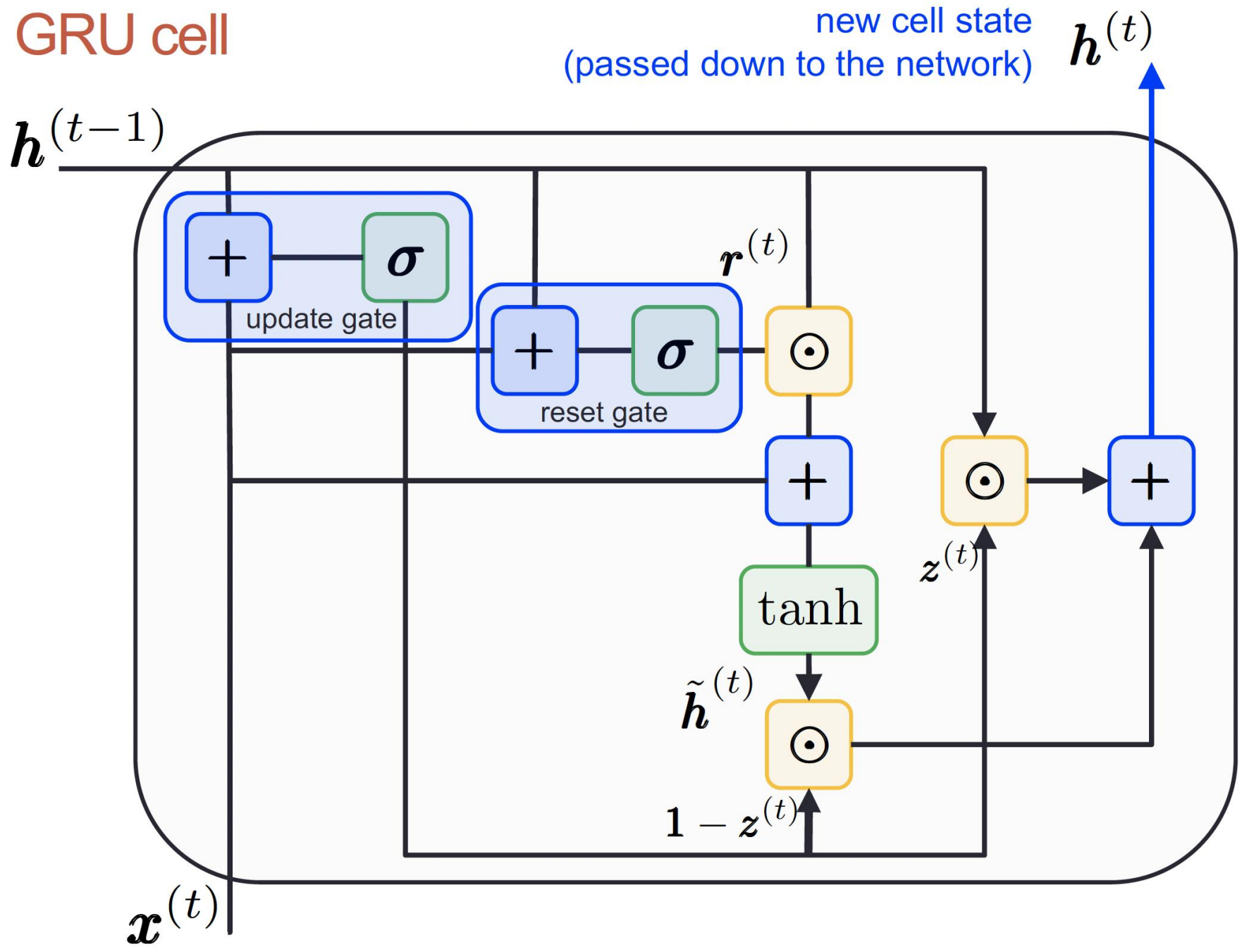


Reference from theory

- LEARNING FOR SEQUENTIAL DATA: TOOLS AND APPLICATIONS

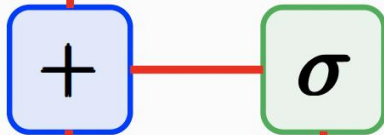


GRU cell



Step 1: update gate

$h^{(t-1)}$



Update gate

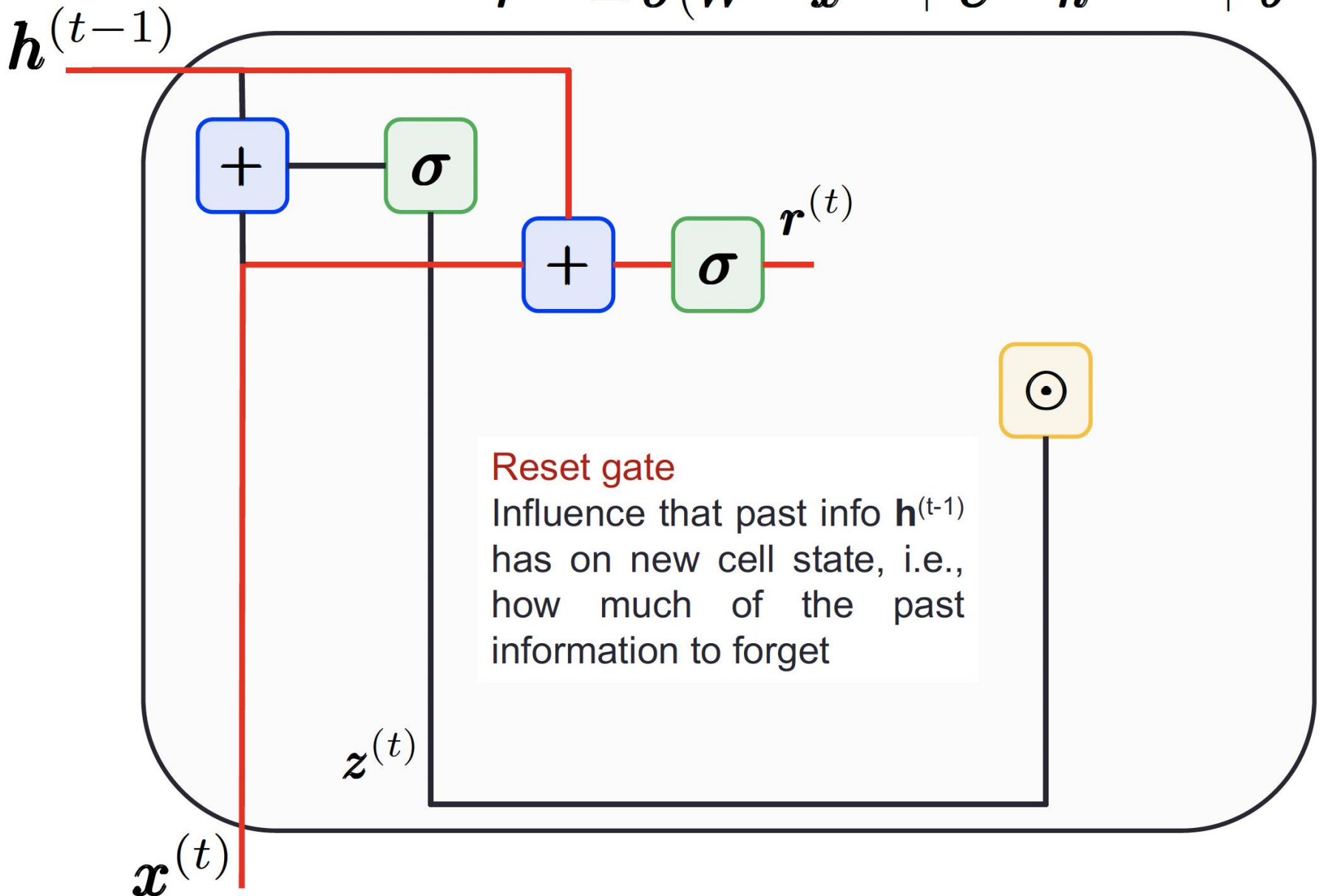
- Controls how much of the past information (from previous time steps) needs to be passed along to the future
- Through it the model can decide to copy all information from the past and remove the problem of **vanishing gradients**
- We will use it at the end...

$x^{(t)}$

$$z^{(t)} = \sigma(W^{(z)}x^{(t)} + U^{(z)}h^{(t-1)} + b^{(z)})$$

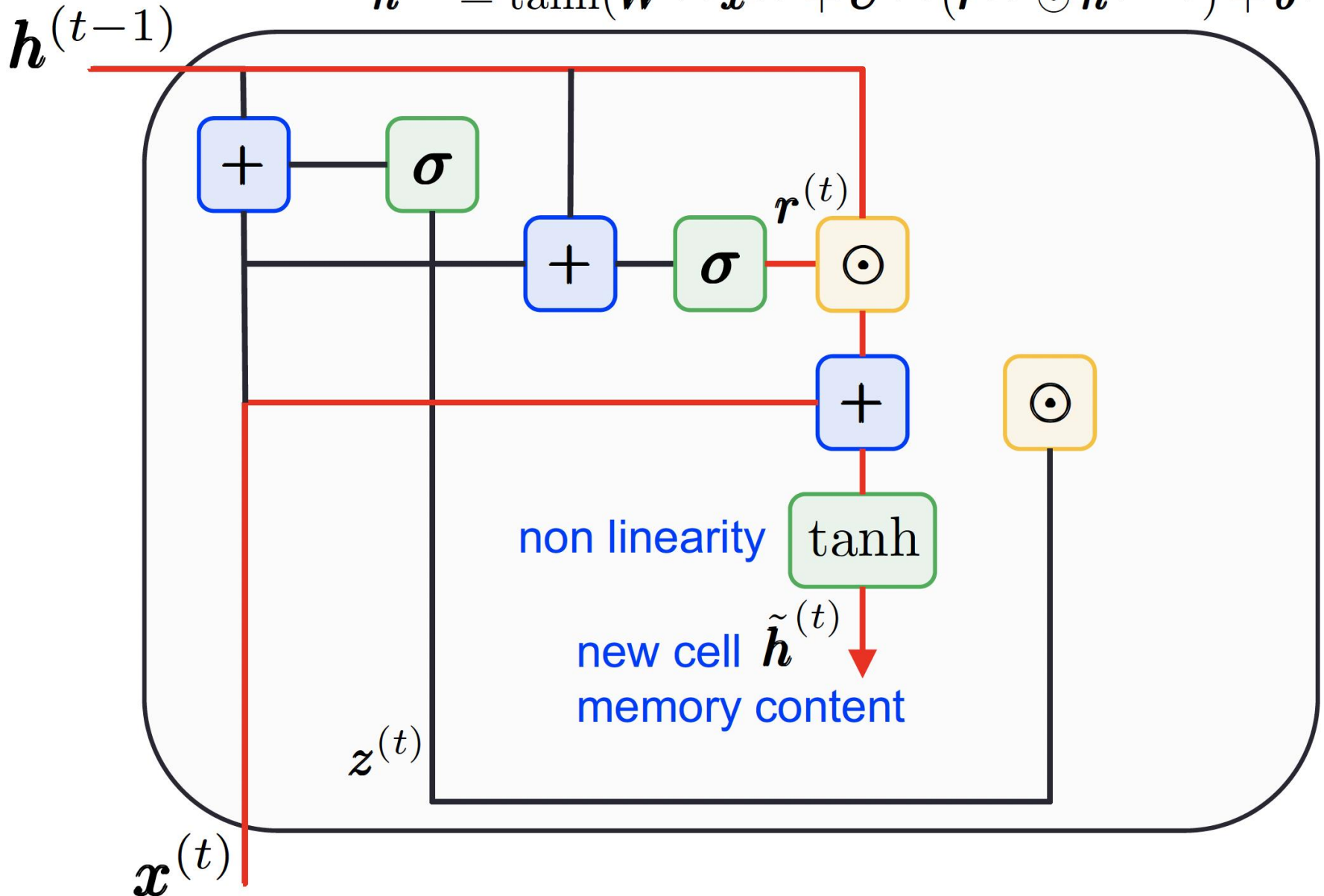
Step 2: reset gate

$$\mathbf{r}^{(t)} = \sigma(\mathbf{W}^{(r)} \mathbf{x}^{(t)} + \mathbf{U}^{(r)} \mathbf{h}^{(t-1)} + \mathbf{b}^{(r)})$$



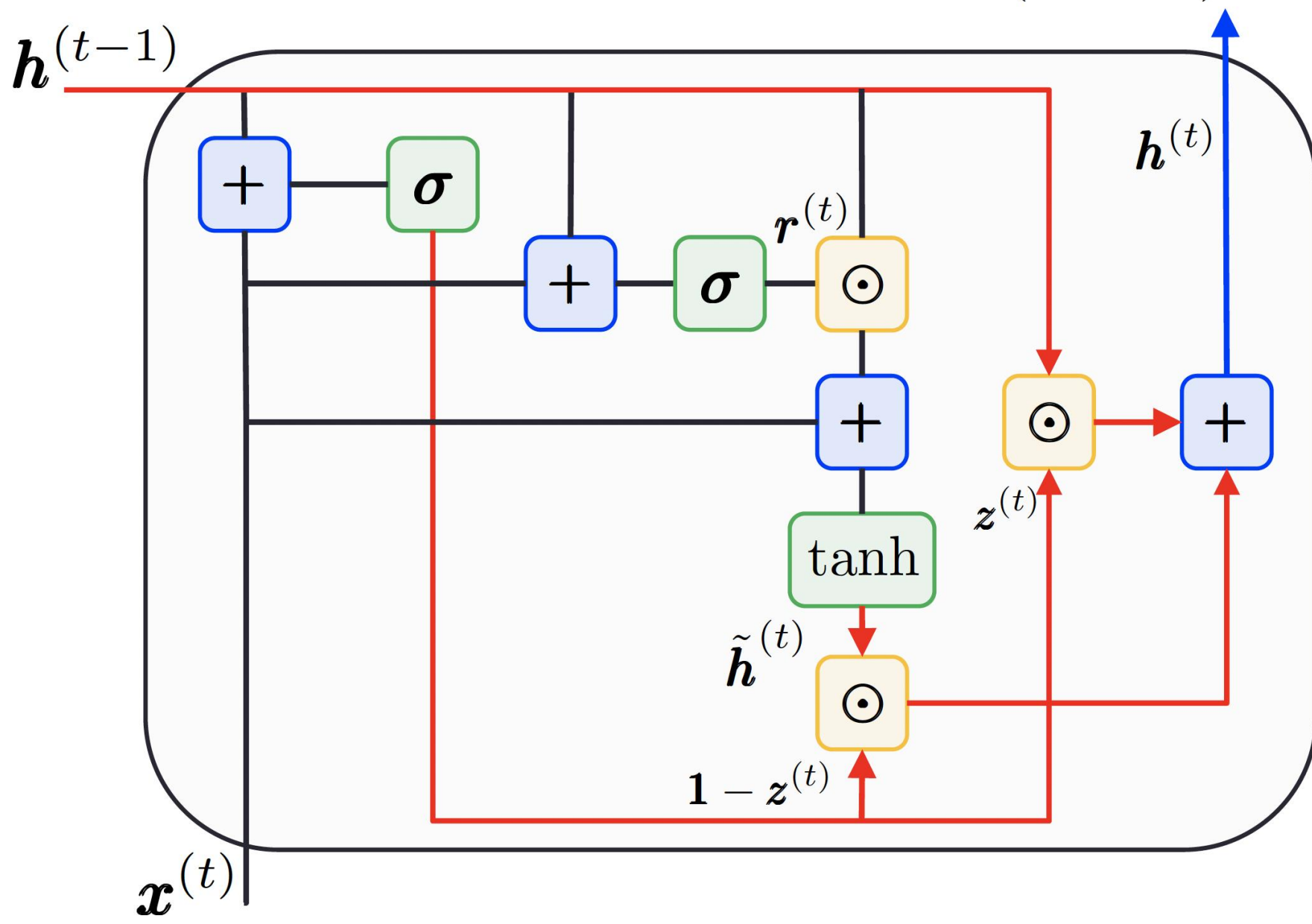
Step 3: current memory content

$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}^{(h)} \mathbf{x}^{(t)} + \mathbf{U}^{(h)} (\mathbf{r}^{(t)} \odot \mathbf{h}^{(t-1)}) + \mathbf{b}^{(h)})$$



Step 4: final state

$$\mathbf{h}^{(t)} = \mathbf{z}^{(t)} \odot \mathbf{h}^{(t-1)} + (1 - \mathbf{z}^{(t)}) \odot \tilde{\mathbf{h}}^{(t)}$$



LAB 5: CNN-RNN CLASSIFIER

Eleonora Cicciarella, Cesare Bidini

eleonora.cicciarella@phd.unipd.it

cesare.bidini@phd.unipd.it

University of Padova, IT

